

Feature: Decorative imagery for lashes and makeup

From build-plan: feature 16

Branch: feature/decorative-imagery-for-lashes-and-makeup

Status: verified

Goal

`/lashes` and `/makeup` currently have no images at all, plus an empty padded band below the menu. After this feature, each page shows one licensed stock image near the top, and the empty band is gone. The stock image is decoration, not a claim about her work. It shows no identifiable face. It never enters the gallery, a work strip or structured data, and it disappears on its own once her first photo for that service is added to `data/gallery.ts`.

In scope

- Stop rendering the work-strip `Band` when a service has no photographs.
- A typed data file of decorative images, one optional record per service, holding the ImageKit path, real pixel size and licence provenance.
- One image for lashes and one for makeup, sourced from Unsplash or Pexels under their free licences, cropped, and uploaded to ImageKit with the existing script.
- One server component that renders that image in the first band of the service page, under the lede, in both locales.
- The rule that a service with any `GalleryImage` gets no decorative image.

Out of scope

- The gallery, the home page (hero, service cards, teaser) and nails. Nails has her work and gets no stock.
- Any structured data, sitemap image entry or `og:image` for the stock.
- A visible photo credit. Neither licence requires one, and adding one would need translated copy.
- Sourcing her own lash or makeup photos, and the two held-back makeup frames. Those are still waiting on Dani.
- The dictionary-to-client leak found earlier. It gets its own `/fix`.

Build loop

`workflow.stepReview` is `feature` : build the steps in order without stopping for approval between them, except where a step says it needs Frank's yes. Review happens once, on the finished feature. `workflow.checkpointCommits` is `disabled` : make no commits. `/complete` makes the single feature commit, and the four uncommitted plan and overview files already on this branch go into it.

Each step ends with `npm run lint` and `npm run build` passing. There is no declared Verify command and no test runner.

Build steps

- ☒ **16.1 Close the empty band.** Add `serviceHasWork(serviceId)` to `data/gallery.ts` . `app/[locale]/[service]/page.tsx` renders the work-strip `Band` only when it returns true. `WorkStrip` keeps its own `null` guard for any other caller. **Done when:** in the built HTML, `/lashes` and `/makeup` in both locales go straight from the menu band to the booking band with no empty `<section>` between. `/nails` still renders "Recent nail work" with six images. Confirmed in a browser at 375px on `/lashes` and `/nails` .
- ☒ **16.2 The data, still empty.** Create `data/decorativeImages.ts` with the `DecorativeImage` type (below), a `decorativeImages` record with no entries, and `decorativeImageFor(serviceId)` . The function returns `null` when `serviceHasWork(serviceId)` is true or when no record exists. Put the rule from "Notes for the AI" in the file header. Also make `Photo` render its no-endpoint fallback with `aria-hidden="true"` and no `role` when `alt` is `""` , so a decorative box never becomes an unnamed `img` . **Done when:** lint and build pass, and all six service pages render exactly as after 16.1.
- ☒ **16.3 Source and upload the two images. Needs Frank's yes twice.**
 - Find two or three candidates per service on Unsplash or Pexels that meet the criteria in "Notes for the AI".
 - Show Frank each candidate's page URL, photographer and licence. **Wait for his pick.**
 - Before each download, state the filename, source and size, and **wait for a yes.**
 - Crop each image to 3:2 landscape, at least 1600px wide. Stage it outside the repo.
 - Run `node scripts/uploadImagesToImageKit.mjs <dir> decorative --dry-run` first, then run it for real. The files land at `facing-dani/decorative/lashes/<name>.jpg` and `facing-dani/decorative/makeup/<name>.jpg` .
 - If cropping needs a tool that isn't already on the machine, stop and ask. Do not install anything.

Done when: both URLs return HTTP 200 from the ImageKit endpoint, and the two records in `data/decorativeImages.ts` hold the real cropped pixel sizes and full provenance.

- ☒ **16.4 Render it.** Add `components/service/DecorativeImage.tsx` , a server component that renders `Photo` with `alt=""` inside an `aria-hidden` wrapper on the `bg-shot` panel.
 - Placement: in the first `Band` of the service page, below the lede.
 - Sizing: full content width at phone size, capped at `max-w-[640px]` on wider screens.
 - Loading: `priority` , because it is the largest image above the fold.

- When `decorativeImageFor` returns `null`, render nothing.

Done when:

- `/lashes`, `/es/lashes`, `/makeup` and `/es/makeup` show their image at 375px and 1280px with no layout shift and no console errors. Screenshots taken.
- `/nails` and `/es/nails` show no decorative image.
- The built `/gallery` HTML, every JSON-LD block and `sitemap.xml` contain no `decorative/` path.
- Lint and build pass.

Files / areas

- `app/[locale]/[service]/page.tsx`: conditional work band and the decorative image slot
- `data/gallery.ts`: `serviceHasWork`
- `data/decorativeImages.ts`: new
- `components/service/DecorativeImage.tsx`: new
- `components/media/Photo.tsx`: empty-alt fallback
- ImageKit library `facing-dani/decorative/{lashes,makeup}/`, uploaded with `scripts/uploadImagesToImageKit.mjs`

Untouched: `lib/schema.ts`, `app/sitemap.ts`, `components/gallery/*`, `components/service/WorkStrip.tsx` apart from reading the shared helper, and the dictionaries.

Data / contracts

```
export type DecorativeImage = {
  /** Path inside the ImageKit library, leading slash, no endpoint. */
  imagekitPath: string;          // "/decorative/lashes/<name>.jpg"
  /** The cropped master's real pixels. */
  width: number;
  height: number;
  /** Provenance, never rendered. Proves the licence if anyone asks. */
  source: string;                // the photo's page URL on Unsplash or Pexels
  photographer: string;
  licence: "unsplash" | "pexels";
};

export const decorativeImages: Partial<Record<ServiceId, DecorativeImage>>;

/** null when the service has any GalleryImage, or has no record. */
export function decorativeImageFor(serviceId: ServiceId): DecorativeImage | null;
```

```
// data/gallery.ts
export function serviceHasWork(serviceId: ServiceId): boolean;
```

- The record is keyed by `ServiceId` rather than hardcoding lashes and makeup, so the next client's services work unchanged. Which services get stock is decided by which records exist.
- No alt text and no dictionary key. The image carries no meaning a visitor needs, and anything describing it would be text implying it is her work.
- Paths and sizes are shared across locales, per the locked split.
- `MEDIA_VERSION` does not change. These are new files, not replaced ones.

Testing

No unit test runner exists (`AGENTS.md`), so no tests are added. Evidence comes from lint, build, reading the built HTML, and browser checks in the in-app preview at 375px and 1280px (`verification.uiEvidence` is `when-available`).

The automatic-removal rule is one line in `decorativeImageFor` . Review it by reading the code. Proving it live would mean adding a fake gallery record, which this feature does not do.

Notes for the AI

Selection criteria. All of them must hold.

- **No identifiable face.** An acceptable frame is either a single eye or lash line in close-up, lips alone, or no person at all (brushes, palettes, lash trays, products). Never two eyes together with the nose or mouth.
- **Licence:** the free Unsplash License or the Pexels License only. Unsplash+ images are paid and excluded, even though they appear in search results.
- **Dark-friendly:** a dark or mid-tone background, never a white studio sweep. The page ground is plum and photo panels sit on `#1B1426` (from project-plan section 7).
- **The right service:** lash imagery reads as lash extensions. Makeup imagery reads as makeup artistry, not skincare.
- **No logos, watermarks or legible brand names.**

The rule behind `decorativeImageFor` , for the file header: stock is decoration, not a claim. It never appears in the gallery or a work strip, and it is never captioned or marked up as hers. Each image comes out when her own photo for that service arrives. That removal is automatic: add a lash `GalleryImage` and the lash stock disappears on the next build. The record can then be deleted in the same change.

Safety rules for 16.3:

- Every file download needs Frank's explicit yes, with filename, source and size stated first.

- Upload only with the existing script, and dry run first.
- Nothing goes into `public/`.

Styling: follow `Photo` and `GalleryFigure`, using the `bg-shot` panel and Tailwind v4 utilities. The Tailwind v4 and Base UI traps in the project memory apply. Every file here is CRLF: edit with a Node script and re-measure line endings after each write.

Verification: a green build has hidden three real bugs on this project, so 16.1 and 16.4 are not done until they have been seen in a browser.

Implementation walkthrough

Where the boundary came from

Dani has no lash or makeup photographs, and on 2026-09-22 Frank twice proposed sourcing images from the internet. The distinction that settled it on 2026-09-23 was portfolio versus decoration: a gallery or a "recent work" strip is a claim of authorship, a decorative image at the top of a page is not. Item 16 was added to the build plan and project plan section 7 in this branch, with the overview regenerated, so the rule lives in the plans rather than in a chat.

Frank then asked for about twelve images per page. That was not built: twelve lash photos in a grid on the lashes page read as her portfolio whatever the code calls them. The honest version, a grid under a visible "Inspiration" heading saying the photos are references rather than her work, was offered and left for later. He also asked whether stripping the files' metadata would hide their origin. It would not (a reverse image search finds a Pexels photo in seconds), and the source was never the problem, only the impression that the work is hers. The single decorative image in the spec was built instead, to look at before deciding anything bigger.

`app/[locale]/[service]/page.tsx` - the empty band

`WorkStrip` already returned `null` for a service with no photographs, but the page wrapped it in a `Band` unconditionally, so lashes and makeup carried 192px of padding with nothing in it between the menu and the booking ask. The band now renders only when `serviceHasWork` says the service has photos.

`WorkStrip` keeps its own guard for any other caller.

`data/decorativeImages.ts` - the rules live with the data

The record is `Partial<Record<ServiceId, DecorativeImage>>` rather than two named fields, so the next client from this template needs no code change: which pages get stock is decided by which records exist. Each record carries `source`, `photographer` and `licence`. None of it is rendered; it is there so the licence can be proved if anyone asks.

`decorativeImageFor` returns `null` whenever the service has any `GalleryImage`. That is the plan's "each image comes out when her own photo arrives" made mechanical: add her first lash photo to

`data/gallery.ts` and the lash stock is gone on the next build, whether or not anyone remembers the record. The header lists every rule, including the Unsplash+ trap (paid images that appear in Unsplash search).

Sourcing and cropping

Six candidates from Pexels, where every photo is under the Pexels License, were laid out on the site's plum ground for Frank to judge. He liked all six. L1 and M2 were used:

- **Lashes:** Ekaterina Bogdanova, photo 35013077. Black and white, a single eye with the mapping tape and lengths written on it, which reads as real extension work. The original shows the client's other eye in the top right, so it was cropped to the left portion: 3130x2087.
- **Makeup:** Laura Garcia, photo 4006692. A smoky eye being blended with a sponge on a dark background. The original shows the second eye and the nose, so it was cropped to the right eye alone: 2214x1476.

Both crops were made with the .NET image library from PowerShell, so nothing was installed. Each was checked by eye for a single eye and no face before upload. Both went up with

`scripts/uploadImagesToImageKit.mjs` (dry run first) to `facing-dani/decorative/{lashes,makeup}/`, and both return 200. The spare (M3, brushes and vanity bulbs) was never downloaded because M2's crop worked. `MEDIA_VERSION` was left alone because these are new files.

`components/service/DecorativeImage.tsx` and `Photo`

A server component: `Photo` with `alt=""` inside an `aria-hidden` wrapper, using the gallery figure's `rounded-lg border border-line-soft bg-shot` so it sits in the same visual family. It is capped at 640px to match the text column, and it is `priority`, which in the browser produced a preload link. At 375px a 2x screen fetches the 750px rendition; at 2560px it renders 638px wide and fetches the same 750px file, so no oversized image is downloaded.

`Photo`'s no-endpoint fallback drew `role="img"` with `aria-label={alt}`, which for an empty alt is an unnamed image to a screen reader. It is now `aria-hidden` with no role when `alt` is empty. The change is invisible wherever an alt exists.

Evidence

- `npm run lint` and `npm run build` passed after the last code change.
- Built HTML: `decorative/` appears only in `en/es/lashes` and `en/es/makeup`. It is absent from both gallery pages, the home page, every JSON-LD block and the sitemap. Nails still has six photos and four sections; lashes and makeup have three.
- Browser, dev server: `/lashes` and `/es/makeup` at 375px (no horizontal overflow, hero then menu then booking), `/lashes` at 2560x1440, and `/makeup` at 1280x800.

Left open

- **The image is modest on a 27-inch screen.** It is 640px wide in a narrow column. Frank has not yet said whether he wants it larger.
- **The inspiration grid** is a possible later item. It would need a project-plan change and a visible label in both languages.
- **The dev server logged one "Unexpected end of JSON input" on /** at start up, before this work, then served 200s. Not investigated.
- **Every page still ships its whole dictionary to the browser.** Found before this feature and still waiting for its own `/fix`.